

CSI243 Functional Programming

Lab 5 Creating Lists from Lists

Objectives

- 1. Lists**
- 2. List Comprehensions**

Background / Scenario

List comprehension allows a list to be described using existing list(s). This is a distinct feature of functional programming languages. As a functional programming language, Haskell provides a syntax for list comprehension.

A list comprehension has the form

```
[output function | qualifiers separated by commas]
```

where each qualifier can either be one of the following:

- A generator – in the form `p <- listExpression`
- A test – which is a boolean expression

Multiple generators allow us to use elements from two or more lists.

As usual, use the information to work on the lab.

For each of the functions, there should be five parts:

- i.** Specify the signature of the function, that is, the arguments and the return type the function
- ii.** Write specifications for the function
- iii.** Dry run the function
- iv.** Implement using Haskell
- v.** Test the function

1. Create a list comprehension that, given a list of integers between 1 and 79, it returns the square of an integer, only if the integer is odd.
2. Assume we have the function `isVowel :: Char -> Bool` which returns `True` if a character is a vowel, and `toUpperCase` which converts a character to upper case. Given the `alphabet= ['a', 'b', 'c', ..., 'x', 'y', 'z']`, produce the list of upper case consonants (not vowels).
3. Three positive numbers form a Pythagorean triple if the sum of the first two numbers is equal to the sum of the square of the third.

As an example (3, 4, 5) are a Pythagorean triple because

$$3^2 + 4^2 = 5^2$$

$$\Rightarrow 9 + 16 = 25$$

Create a list comprehension that returns triples (tuples with three elements) for Pythagorean triples below 100. Mathematically, there are 16 primitive Pythagorean triples of numbers up to 100, so your comprehension should return 16 triples. This is because (3,4,5) and (4,3,5) are the same thing. Ensure your generators are set properly.

4. Assume you are given a list of student marks in the format `[T1, T2, A1, A2, A3]`, where T represents a test mark and A represents an assignment mark. Your task is to create a list comprehension that will return a tuple that contains the average mark for the tests and the assignments for each student. For example, assume you have the following list called `marks`
`[[20, 35, 67, 89, 11], [45, 78, 90, 12, 34], [90, 67, 89, 56, 78]]`,
then comprehension should return
`[(27.5, 55.7), (61.5, 45.3), (78.5, 74.3)]`.

=====

SUBMIT BEFORE THE END OF THE LAB ON THE LINK ON MOODLE